Modelling Human Activities using Hidden Markov Models

1.Background and Motivation

Use Case: Affordable way to monitor activities of elderly individuals living alone using their existing Smartphone. An activity monitoring wearable would be too costly for most Rwandan households. Their phone can however be used for free. This can be done by using the accelerometers and gyroscopes on the phone to identify if the person is walking, stationary, doing a sudden high intensity activity (in this case jumping). If there is a long period of stationary activity or sudden periods of intense activity followed by long rest periods a relative/community health worker may call to check on them. The activity information is hidden within the noisy data which can be modelled using a Hidden Markov Model (HMM).

2. Data collection and pre-processing

Item	Value
Participant and device	1 participant; iPhone 13, iOS 18.5, Sensor Logger app
Sampling rate	100 Hz for all sensors (measured between 100.21 Hz and 100.23 Hz)
Sensors	accelerometer, gyroscope, uncalibrated accelerometer (includes gravity)
Training data	52 files: 13 per activity, 6.8–13.6 s each; 120–139 s per activity
Unseen test data	test1 (41.4 s), test2 (40.5 s): all 4 activities, about 10 s each

The videos show the phone held at waist height while standing, walking and jumping. For still images, the phone was held at an angle of approximately 45°.

videos

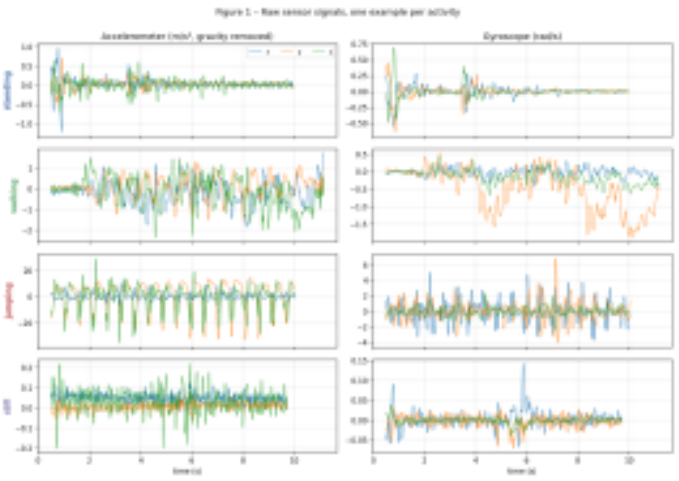
Stand:https://drive.google.com/file/d/1UKD52YU7C0L3poasf7u_e04lLHbmcW3m/view?usp=sharing

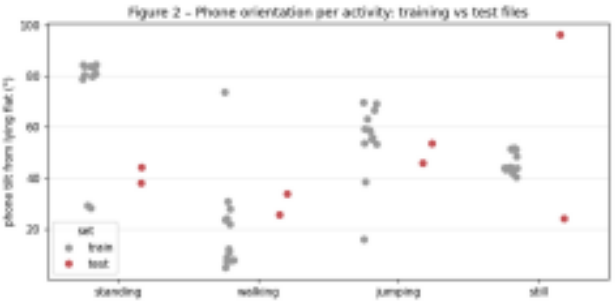
Walk:https://drive.google.com/file/d/1ZHkZcLfCg2Lf940V_0hk3iIpSlcFqOZX/view?usp=sharing

Jump:
https://drive.google.com/file/d/1FdOV-0T179y4UlS8zeaEBTlB2ccJeBir/view?usp=sharing

Still:https://drive.google.com/file/d/16zcTTNA5F_wVan2hEIKBkJTVpaWfMe8r/view?usp=sharing

Data harmonisation/cleaning:
As only one phone was being used there was a slight difference in timestamps between the 3 signals. The sampling frequency was also drifting (around 100.2Hz). As such all the recordings were resampled to exactly 100Hz using linear interpolation. A duplicate export was removed, and 0.5 s was trimmed from both ends of each training file to remove the Start/Stop tap. The cleaned files, with timestamp and label columns, are in `data/train` and `data/test` .





Windowing: The signals were divided into time segments of 2 s (200 samples) with 50% overlap between them. As the steps and jumps will be around 1.5-3.5Hz there will be multiple complete cycles in each segment. There will also be a high resolution of 0.5Hz as there are 200 samples (sample rate = 100). This resulted in 400 training samples and 79 testing samples.

The features were calculated using the magnitude of each sensor. This is calculated using √(x2+y2+z2). The magnitude won't change if the phone is rotated in the hand.

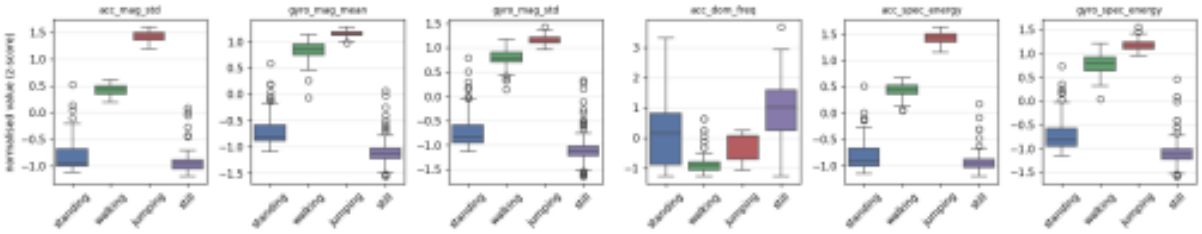
Feature	Domain	Why it helps
Std of acceleration magnitude	Time	Movement intensity: jumping > walking > standing> still
Mean rotation rate	Time	Body sway of a hand-held phone; the main clue for standing vs still
Std of rotation rate	Time	Rhythmic swinging vs steady
Dominant frequency (FFT)	Frequency	Rhythm: walking about 1.5 Hz; standing and still show only noise
Acceleration spectral energy (FFT)	Frequency	Vibration energy, separating jumping from walking

Rotation spectral energy (FFT)	Frequency	Separates low motion from no motion

Mean accelerations, amplitude of the signal and phone orientation were excluded by choice. For a mostly static phone these variables primarily represent bias and orientation. Orientation varied between sessions (Figure 2).

Normalisation: A log transform was applied to the 5 skewed intensity and energy features so that the distribution of each class is approximately Gaussian. All features were then Z-scored using only training statistics so that no information was included from the test data (Figure 3).

Figure 3 - Normalised features by activity (training data)



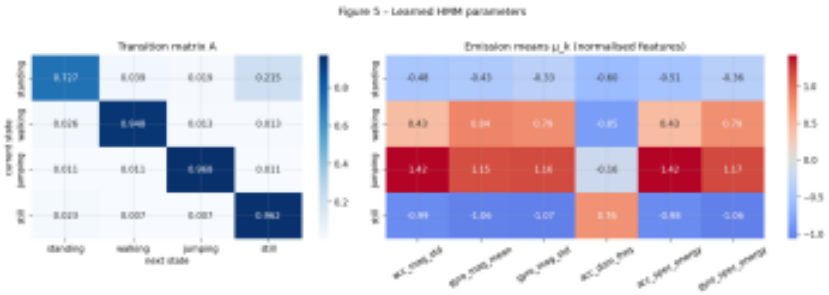
3. HMM setup and implementation

Element	Definition
Hidden states Z	standing, walking, jumping, still
Observations X	6-dimensional normalised feature vector per 2 s window
Transitions A	4×4 matrix, P(state at t+1 given state at t)
Emissions B	one diagonal Gaussian per state, N(x; μ□, diag(σ□²)), suited to continuous features
Initial π	P(first window is state k)

Algorithms: The Baum-Welch algorithm (EM using forward-backward) is used to learn the parameters π , A and Gaussians by maximising the training data likelihood. The Viterbi algorithm finds the most likely sequence of activities for new data. Using the emissions and transitions we can smooth out noisy single windows, which should improve sensitivity, specificity and accuracy. The log-likelihood can be found using the forward algorithm.

Training: We initialise the emission parameters using the labelled windows for each activity. We set π to be uniform, A to be 0.9 self transition probabilities (activity last around 10 windows meaning there is a probability of around 1 - 1/10 of staying in the same state). The Baum-Welch algorithm is run until there is less than an improvement of $\varepsilon = 10^{-4}$ on the objective function. After 9 iterations it had converged. The log-likelihood increased from 144.5 to 498.3. Convergence and emission density plots can be seen in the notebook. The self transitions of the learned A matrix are 0.95-0.97 for walking, jumping and still. Standing

has a self transition of 0.73 and transitions to still with probability 0.22.

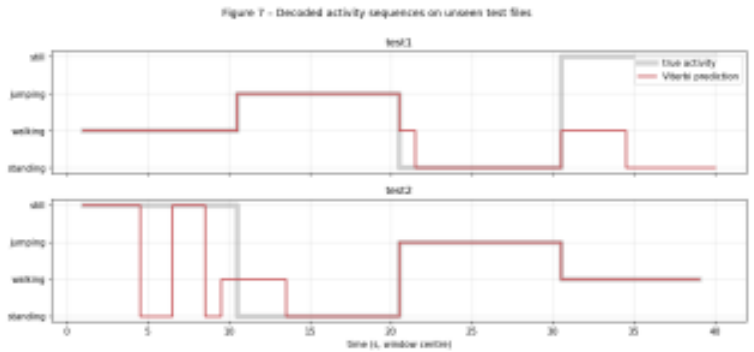


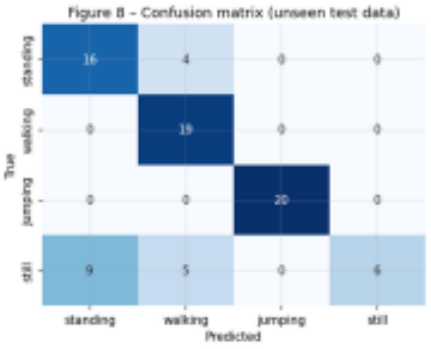
4. Results and Interpretation:

Unseen Data. The data in test1 and test2 was recorded in a separate session once all training had been carried out. These tests consisted of the four activities performed back to back in different order (test1: walk, jump, stand, still and test2: still, stand, jump, walk). These tests were never used for training or normalisation. The transitions are real life and the phone was held at different angles. The overall accuracy was 77.2% (61/79).

State (Activity)	Number of Samples	Sensitivity	Specificity	Accuracy (one-vs rest)	Overall Accuracy
Standing	20	0.800	0.847	0.835	0.772
Walking	19	1.000	0.850	0.886	0.772
Jumping	20	1.000	1.000	1.000	0.772
Still	20	0.300	1.000	0.823	0.772

The jumping and walking classes are easily identified. However, only 6 windows were correctly classified (9 windows labelled standing, 5 windows labelled walking). It has a high specificity of 1.00 since no false predictions were made meaning it is conservative. The walking class had a lower specificity of 0.85 due to it being falsely classified in 9 windows. Many of these were during transitions. For example in Figure 7 the still section of test1 can never be identified. When the phone was placed down it was decoded as walking and when the phone was still it was decoded as standing. However, in test2 still was identified for approximately 6 seconds but with some flickering between walking and standing.





Model	Overall accuracy

Emissions only (each window alone)	0.797
Viterbi, initial parameters	0.759
Viterbi after Baum–Welch	0.772

The Baum-Welch improved the classification accuracy by 1.3 points over the Viterbi algorithm, although the window based classification was slightly better. This is because there are no transitions in the training files, meaning that the high self transitions result in a slow changing model that can misclassify for extra windows.

5 Discussion and Conclusion

Easiest and hardest: The easiest classes to classify were jumping and walking as they have very different intensities and rhythms to the rest. They also withstood the new phone angle. The hardest to classify were standing and still. There are only small clues to distinguish between the two as the only thing moving is the swaying of the hands (mean rotational velocity = 0.022 compared to 0.011 rad/s in training). This difference is less than the noise between the sessions.

Transition Probabilities: The high self transitions (0.95-0.97) are realistic as people will do an activity for a long period of time. However as the training data only had one activity, the off-diagonal values are very small as there are very few real transitions. The 0.22 transition from standing to still is not realistic, it shows that the Baum-Welch algorithm is mixing the two states and isn't able to completely separate them.

Noise, Orientation and Sampling Frequency: The sampling frequency of 100Hz was sufficient as the energy of the movement is below 5Hz which is much lower than the Nyquist frequency (50Hz). Also, only one phone was used meaning there were no issues with sampling frequencies. Orientation was more important, the still phone in the test session was at around 24 degrees and 96 degrees (sideways) compared to 45 degrees. Without using orientation, walking and jumping were robust, however we lost the most obvious clue between standing and still. Windows that cross over a transition would be classifying a mix of activities and usually classified as walking.

Improvements: Collect training data from a variety of angles and positions (held in the hand, pocket and on a table). Include transitions in the training data so that Baum-Welch can learn realistic transitions. Use more participants and phones. Add a proximity/touch sensor to tell if the phone is being held or on a table. Use a Gaussian Mixture for each state.

Conclusion: A Gaussian HMM was created from scratch using six orientation independent features. Walking and jumping were classified perfectly. Overall, the model achieved an accuracy of 77.2% on unseen data. The issue with standing and still phone is due to the lack of training data from one angle, not the algorithms. This is the next thing to improve for elder monitoring.

Repository: https://github.com/Larissa4-droid/hmm-recognition